

Micro-lab — Computational cost

Friday, October 30

Last updated: October 8, 2026.

Date is on the [schedule](#). Same day: short [Lecture 29](#) (Boyd's $ab^T c$ example), then this micro-lab. Precursor to the [capstone](#): same math, different cost.

Credit: Lab points are for effort in class, not for a Canvas file upload. Work in Octave or Python on a laptop (demos and both-language code are on this hub). On lab day you get a **self-contained one-pager** with the exercises written out, space for answers and new syntax, and a blank for instructor initials after a short check-in. The [one-pager](#) is posted so you can preview it — write on the hard copy handed out in class. Quizzes will ask about these ideas ([Quiz 8](#) covers Week 10).

Resources: [Syntax sheets](#) · [Resources](#) · [Lecture 29](#) · Boyd VMLS (matrix multiplication / complexity discussion around $ab^T c$).

What we are doing

Compute $y = (ab^T)c$ two ways: form the outer product first, or multiply the scalar $b^T c$ first. Check they agree, count storage, time both for growing n , and justify which order to use.

Learning objectives

1. Compare equivalent computation orders.
2. Count storage (intermediate values).
3. Measure runtime for increasing problem sizes.
4. Justify an order quantitatively.

Math connections

Associativity $(XY)z = X(Yz)$ and the identity $(ab^T)c = a(b^T c)$ from [Lecture 29](#); products from [Lecture 27](#). Capstone will again ask: same answer, cheaper path?

Key syntax (this lab)

Task	Octave	Python
Column vectors	<code>a = randn(n,1);</code>	<code>a = np.random.randn(n)</code>
Outer product ab^T	<code>a*b'</code>	<code>np.outer(a, b)</code>
Matrix–vector	<code>M*c</code>	<code>M @ c</code>
Inner product $b^T c$	<code>b'*c</code>	<code>b @ c (1D) or b.T @ c</code>
Scale vector by scalar	<code>s*a</code>	<code>s * a</code>
Wall time	<code>tic; ...; toc</code>	<code>time.perf_counter()</code>

Indexing reminder: Boyd x_1 is 1-based. Octave is 1-based; NumPy is 0-based. Today's work is mostly whole-vector ops.

Demo (optional — run once)

Small n check that both orders match.

Octave:

```
a = [1; 2]; b = [3; 4]; c = [1; -1];
y1 = (a*b')*c
y2 = a*(b'*c)
```

Python:

```
import numpy as np
a = np.array([1.0, 2.0]); b = np.array([3.0, 4.0]); c = np.array([1.0, -1.0])
y1 = np.outer(a, b) @ c
y2 = a * (b @ c)
print(y1, y2)
```

Expected output:

```
-1
-2
```

(both orders; Python prints something like `[-1. -2.]` twice).

Exercises

Pick Octave or Python for the term. Use the **same random** a, b, c when comparing two orders for a fixed n .

E1. Tiny hand check. With $a = (1, 0, -1)$, $b = (2, 1, 0)$, $c = (1, 1, 1)$, compute by hand (or tiny code) both $(ab^T)c$ and $a(b^Tc)$.

Expected answer: both equal $(3, 0, -3)$ (since $b^Tc = 3$).

E2. Storage. For general n , how many entries are stored in the intermediate $M = ab^T$? How many numbers is the intermediate scalar $s = b^Tc$? For $n = 2000$, write both counts.

Expected answer: n^2 vs 1; for $n = 2000$: 4,000,000 vs 1.

E3. Agree for larger n . Generate random n -vectors with $n = 500$. Compute

$$y_1 = (ab^T)c, \quad y_2 = a(b^Tc)$$

and report a measure of agreement (e.g. `norm(y1-y2) / np.linalg.norm(y1-y2)`).

Octave:

```
n = 500;
a = randn(n,1); b = randn(n,1); c = randn(n,1);
y1 = (a*b')*c;
y2 = a*(b'*c);
norm(y1-y2)
```

Python:

```
import numpy as np
n = 500
rng = np.random.default_rng(0)
a = rng.standard_normal(n); b = rng.standard_normal(n); c = rng.standard_normal(n)
y1 = np.outer(a, b) @ c
y2 = a * (b @ c)
print(np.linalg.norm(y1 - y2))
```

Expected output: a tiny number (order 10^{-10} or smaller — floating-point noise, not a modeling error).

E4. Time both orders. For $n = 200$, $n = 1000$, and $n = 3000$ (skip 3000 on a slow machine if needed), time Order 1 $(a*b')*c$ / $\text{np.outer}(a,b)@c$ and Order 2 $a*(b'*c)$ / $a*(b@c)$. Record times on the ticket.

Octave pattern:

```
n = 1000;
a = randn(n,1); b = randn(n,1); c = randn(n,1);
tic; y1 = (a*b')*c; t1 = toc;
tic; y2 = a*(b'*c); t2 = toc;
t1, t2
```

Python pattern:

```
import numpy as np, time
n = 1000
rng = np.random.default_rng(1)
a = rng.standard_normal(n); b = rng.standard_normal(n); c = rng.standard_normal(n)
t0 = time.perf_counter(); y1 = np.outer(a, b) @ c; t1 = time.perf_counter() - t0
t0 = time.perf_counter(); y2 = a * (b @ c); t2 = time.perf_counter() - t0
print(t1, t2)
```

Expected: Order 2 much faster; the gap grows with n . Absolute times vary by machine — report yours.

E5. Justify. In 2–3 sentences on the ticket: which order would you use for $n = 10^4$, and what two quantitative observations (storage count + timing trend) support that choice?

Expected answer (idea): scalar-first — stores 1 intermediate vs 10^8 matrix entries; timings show Order 1 growing far faster (roughly with n^2).

One-pager

Preview the [micro-lab ticket](#); write on the hard copy in class.